SIMATS ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO 1_ASSIGNMENT

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Course Outcome: CO1 – Apply OOP principles and C++ constructs including data types, operators and control structures
Student Name: Keshavaram L
Register Number: 192511253
Department: B.E. Computer Science and Engineering
Academic Year: 2026–2027
GitHub Repository Link: https://github.com/keshavaram0104-boop/DSA0118_OOP-WITH-C-_192511253_KESHAVARAM-L

COURSE OUTCOME

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs.

ASSIGNMENT OVERVIEW

The assignment models a metro smart-card fare system with Inner, Middle and Outer zones. Base fares are Rs 10, Rs 20 and Rs 30. Peak hours are 8–10 AM and 5–8 PM, with a 20% surcharge. A valid journey must leave at least Rs 50 on the card. Money is represented as integer paise to avoid floating-point rounding.

TASK 1 – Understand the Problem and Identify the Objects

The main objects are SmartCard, Passenger, Journey, Zone and FareTable. SmartCard stores a card identifier and private balance; its behaviours include recharge, canTravel and deduct. Passenger stores passenger details and concession eligibility. Journey records source/destination zone, travel time and fare. Zone represents the network area. FareTable stores the base fare for each zone. These objects are justified because each has its own state and responsibility.

In a procedural solution, global variables and free functions calculate fares and update balances. In an OOP solution, data and operations are grouped into classes. Encapsulation protects the balance, abstraction exposes only required operations, inheritance can represent StudentPassenger and SeniorPassenger derived from Passenger, and polymorphism can provide different concession calculations through a common interface. Invariants: balance must never become negative; after a paid journey at least Rs 50 must remain; only valid zones and hours are accepted; fare calculation must use exact paise.

Both procedural and OOP designs take O(1) time per journey and O(1) storage per card. A rule change in a procedural design may require edits across multiple functions; a centralized FareTable or fare-policy class localizes the change.

TASK 2 – Tokens, Keywords, Identifiers and Constants

Tokens are keywords (class, int, return), identifiers (SmartCard, balancePaise), constants (5000, 20), operators (+, -, ==) and punctuators ({ }, ;, ,). constexpr is preferred for compile-time fare values because it is typed, scoped and checked by the compiler. #define performs textual substitution and has no type or scope. const prevents modification at run time after initialization. enum class Zone { Inner, Middle, Outer } gives scoped, strongly typed zone values and avoids accidental mixing with integers. Identifiers should be meaningful, begin with a letter or underscore, and avoid reserved keywords.

Replacing constexpr and enum class with macros weakens type checking and scope safety. Both compile-time constants and macros generally add no per-journey storage and do not materially change run time; constexpr is safer and easier to maintain.

TASK 3 – Data Types and Type Compatibility

Use long long for accumulated paise and large journey totals; int for hour, zone index and percentage; bool for peak/concession flags; enum class for zone; std::string for names and identifiers. Use double only for measurements that genuinely need fractional values, not currency. Integer paise ensures exact arithmetic. sizeof and numeric limits are platform-dependent and should be recorded on the target compiler using sizeof and <climits>. On common 64-bit Linux with g++, int is 4 bytes and long long is 8 bytes.

At 3,000,000 journeys with a maximum fare of Rs 36, the total is Rs 108,000,000 (10,800,000,000 paise). This exceeds a signed 32-bit int in paise, so long long is required. Addition is O(1) for integer and double values; storage is typically 8 bytes for both long long and double. Double may accumulate binary representation error, while integer paise remains exact.

TASK 4 – Control Implicit Conversions

SOURCE CODE

```
#include <iostream>

int main() {
    int a = 7, b = 2;
    double c = a / b;
    double d = static_cast<double>(a) / b;
    std::cout << c << ' ' << d << '\n';

    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';

    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';

    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

OUTPUT

```
3 3.5
C 68
1 true
s >= u
```

Integer division 7/2 produces 3 before assignment to double; casting one operand to double produces 3.5. Character arithmetic promotes char to int, so A+2 prints C and ch+1 prints 68. Assigning 5 to bool produces true, displayed as 1 and then true after boolalpha. Comparing signed -1 with unsigned 1

converts the signed operand to unsigned, making the condition false on the usual unsigned conversion rules. Use static_cast for intentional conversions and brace initialization to reject narrowing.

TASK 5 – Write Correct Expressions with the Right Precedence

SOURCE CODE

```
#include <iostream>

int main() {
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';

    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

    int k = 0;
    for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\n';
}
```

OUTPUT

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

The peak predicate is (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20). The parentheses make the two intervals explicit. Multiplication has higher precedence than addition; parentheses override precedence. Post-increment yields the old value before incrementing, while pre-increment yields the incremented value. The conditional operator selects one of two expressions. Integer division truncates toward zero; the remainder has the sign of the dividend. The loop adds 0, 3, 6 and 9 to produce 18. A 24-entry bool lookup table is also O(1) per check but uses 24 bytes and requires initialization.

TASK 6 – Design the Control Structures

SOURCE CODE

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };
```

```
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return 0;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}

int main() {
    long long balance = 10000;
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };

    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                      << std::setfill('0') << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

OUTPUT

```
Fare Rs 10.00 balance Rs 90
```

```
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Algorithm: validate the zone and hour; select the base fare using switch; if peak, add 20% using integer arithmetic; check whether balance minus fare is at least 5000 paise; deduct only when permitted; report the fare and remaining balance. A switch is clear for a small fixed set of zones, if–else is suitable for range conditions, and a loop processes multiple trips. Invalid entries can be skipped with continue. A default switch branch handles unknown zones.

TASK 7 – From Procedural to Object-Oriented

SOURCE CODE

```
#include <iostream>

constexpr long long MIN_BALANCE = 5000;

bool canTravel(long long balance, long long fare) {
    return fare >= 0 && balance - fare >= MIN_BALANCE;
}

bool deductProcedural(long long& balance, long long fare) {
    if (!canTravel(balance, fare)) return false;
    balance -= fare;
    return true;
}

class SmartCard {
private:
    long long balancePaise;

public:
    explicit SmartCard(long long initialBalance) : balancePaise(initialBalance) {}

    bool canTravel(long long fare) const {
        return fare >= 0 && balancePaise - fare >= MIN_BALANCE;
    }

    bool deduct(long long fare) {
        if (!canTravel(fare)) return false;
        balancePaise -= fare;
        return true;
    }

    long long getBalance() const {
```

```
        return balancePaise;
    }
};

int main() {
    long long proceduralBalance = 10000;
    std::cout << "Procedural: "
              << (deductProcedural(proceduralBalance, 3000) ? "Approved" : "Denied")
              << ", balance paise = " << proceduralBalance << '\n';

    SmartCard card(10000);
    std::cout << "OOP: "
              << (card.deduct(3000) ? "Approved" : "Denied")
              << ", balance paise = " << card.getBalance() << '\n';
}
```

OUTPUT

```
Procedural: Approved, balance paise = 7000
OOP: Approved, balance paise = 7000
```

The procedural version passes balance values to free functions. The OOP version stores balance privately in SmartCard and exposes canTravel() and deduct(). Private data prevents unrelated code from assigning a negative balance directly. Both approaches use O(1) time and O(1) storage. Making balance public does not change complexity, but it weakens encapsulation and permits invariant violations.

TASK 8 – Read and Print Data Safely

SOURCE CODE

```
#include <iomanip>
#include <iostream>
#include <limits>

int main() {
    int hour;
    long long balancePaise;

    std::cout << "Enter hour (0-23): ";
    while (!(std::cin >> hour) || hour < 0 || hour > 23) {
        std::cout << "Invalid hour. Enter an integer from 0 to 23: ";
        if (std::cin.fail()) {
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
```

```
    }

    std::cout << "Enter card balance in paise: ";
    while (!(std::cin >> balancePaise) || balancePaise < 0) {
        std::cout << "Invalid balance. Enter a non-negative integer: ";
        if (std::cin.fail()) {
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
    }

    std::cout << "Balance: Rs " << balancePaise / 100 << '.'
              << std::setw(2) << std::setfill('0') << balancePaise % 100
              << std::setfill(' ') << '\n';
}
```

OUTPUT

```
Enter hour (0-23): Enter card balance in paise: Balance: Rs 123.45
```

The run uses input values 9 and 12345. When extraction fails, cin.fail() detects the error; cin.clear() resets the stream state and ignore() discards the invalid line. setw and setfill format the paise portion with two digits. getline plus stoi is an alternative; both parse a line in O(L), but exception handling can make invalid-input recovery explicit.

TASK 9 – Implement and Test the Fare Logic

SOURCE CODE

```
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>

enum class Zone { Inner, Middle, Outer };
constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr int PEAK_PERCENT = 20;

long long baseFare(Zone zone) {
    switch (zone) {
        case Zone::Inner: return 1000;
        case Zone::Middle: return 2000;
        case Zone::Outer: return 3000;
    }
    return -1;
}
```

```
bool isPeak(int hour) {
    return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}

long long calculateFare(Zone zone, int hour, int concessionPercent = 0) {
    long long fare = baseFare(zone);
    if (fare < 0 || hour < 0 || hour > 23) return -1;
    if (isPeak(hour)) fare += fare * PEAK_PERCENT / 100;
    if (concessionPercent < 0 || concessionPercent > 100) return -1;
    fare -= fare * concessionPercent / 100;
    return fare;
}

bool canTravel(long long balance, long long fare) {
    return balance >= 0 && fare >= 0 && balance - fare >= MIN_BALANCE_PAISE;
}

std::string money(long long paise) {
    return "Rs " + std::to_string(paise / 100) + "." +
           (paise % 100 < 10 ? "0" : "") + std::to_string(paise % 100);
}

int main() {
    struct Test {
        Zone zone;
        int hour;
        int concession;
        long long balance;
        const char* label;
    };

    const Test tests[] = {
        {Zone::Inner, 11, 0, 10000, "Inner off-peak"},
        {Zone::Middle, 8, 0, 10000, "Middle at 8 AM"},
        {Zone::Outer, 19, 0, 10000, "Outer at 7 PM"},
        {Zone::Outer, 20, 0, 10000, "Outer at 8 PM"},
        {Zone::Middle, 10, 0, 10000, "Middle at 10 AM"},
        {Zone::Outer, 12, 0, 8000, "Leaves exactly Rs 50"},
        {Zone::Outer, 12, 0, 7900, "Insufficient balance"}
    };

    for (const Test& t : tests) {
```

```
        long long fare = calculateFare(t.zone, t.hour, t.concession);
        bool approved = canTravel(t.balance, fare);
        std::cout << t.label << ": ";
        if (approved) {
            std::cout << "Approved, fare " << money(fare)
                      << ", remaining " << money(t.balance - fare);
        } else {
            std::cout << "Denied";
        }
        std::cout << '\n';
    }

    std::cout << "Invalid zone, negative recharge and non-numeric hour must be "
              "rejected by input validation before fare calculation.\n";
}
```

OUTPUT

```
Inner off-peak: Approved, fare Rs 10.00, remaining Rs 90.00
Middle at 8 AM: Approved, fare Rs 24.00, remaining Rs 76.00
Outer at 7 PM: Approved, fare Rs 36.00, remaining Rs 64.00
Outer at 8 PM: Approved, fare Rs 30.00, remaining Rs 70.00
Middle at 10 AM: Approved, fare Rs 20.00, remaining Rs 80.00
Leaves exactly Rs 50: Approved, fare Rs 30.00, remaining Rs 50.00
Insufficient balance: Denied
Invalid zone, negative recharge and non-numeric hour must be rejected by input validation before fare calculation.
```

The peak interval is start-inclusive and end-exclusive: 08:00 and 17:00 are peak; 10:00 and 20:00 are off-peak. A balance that leaves exactly Rs 50 is accepted. The test table should include each zone in peak and off-peak periods, all four boundaries, invalid zones, negative recharge and non-numeric input. A loop over t test cases takes O(t) total time and allows new boundary cases to be added as rows.

TASK 10 – Compare Expectations with Execution

For an illustrative constant fare of Rs 10 per journey, totals are Rs 200,000 for 20,000 journeys, Rs 5,000,000 for 500,000 journeys and Rs 30,000,000 for 3,000,000 journeys. In paise these are 20,000,000; 500,000,000; and 3,000,000,000. The last value exceeds signed 32-bit int capacity, so long long is needed. The actual total must be recalculated from the fare mix used in the experiment. Integer addition and double addition are O(1); long long is typically 8 bytes and double typically 8 bytes.

Compile with g++ -std=c++17 -O0 -Wall -Wextra. A signed/unsigned comparison warning indicates potentially surprising conversion behaviour; resolve it by using consistent types or a deliberate checked conversion. Record any other warning emitted by the compiler and explain it rather than suppressing it.

TASK 11 – Recommend and Defend the Final Design

Retain the class-based design, use enum class for zones, constexpr for fixed fares and limits, and long long integer paise for money. Keep the balance private and expose validated operations. Centralize fare policy in one table or policy class. The design is O(1) per journey and O(1) per card; this is suitable for millions of journeys because each journey requires a fixed number of operations. Limitations include compile-time fare changes requiring rebuild and the need to add tests whenever policies change. A configuration file can make fare updates operationally easier at the cost of O(z) start-up parsing and O(z) storage for z zones.

TASK 12 – Re-evaluate and Adapt the Design

SOURCE CODE

```
#include <iostream>
#include <string>

enum class Zone { Inner, Middle, Outer, Airport };
constexpr long long FARES[] = {1000, 2000, 3000, 6000};
constexpr long long MIN_BALANCE_PAISE = 5000;

long long getBaseFare(Zone zone) {
    int index = static_cast<int>(zone);
    if (index < 0 || index >= 4) return -1;
    return FARES[index];
}

class MonthlyPass {
private:
    bool active;
    bool covers[4];

public:
    MonthlyPass(bool enabled, Zone first, Zone second) : active(enabled) {
        for (bool& item : covers) item = false;
        covers[static_cast<int>(first)] = true;
        covers[static_cast<int>(second)] = true;
    }

    bool allows(Zone zone) const {
        return active && covers[static_cast<int>(zone)];
    }
};

class SmartCard {
```

```
private:
    long long balance;

public:
    explicit SmartCard(long long amount) : balance(amount) {}

    bool canPay(long long fare) const {
        return fare >= 0 && balance - fare >= MIN_BALANCE_PAISE;
    }

    bool pay(long long fare) {
        if (!canPay(fare)) return false;
        balance -= fare;
        return true;
    }

    long long getBalance() const { return balance; }
};

long long calculateFare(Zone zone, bool weekend, bool peak) {
    long long fare = getBaseFare(zone);
    if (fare < 0) return -1;

    // Weekend discount and peak surcharge are mutually exclusive.
    if (weekend) {
        fare -= fare * 10 / 100;
    } else if (peak) {
        fare += fare * 20 / 100;
    }
    return fare;
}

int main() {
    SmartCard card(20000);
    MonthlyPass pass(true, Zone::Inner, Zone::Middle);

    const Zone journey = Zone::Airport;
    if (pass.allows(journey)) {
        std::cout << "Journey covered by monthly pass\n";
    } else {
        long long fare = calculateFare(journey, false, true);
        if (card.pay(fare)) {
            std::cout << "Fare paid in paise: " << fare << '\n';
```

```
        std::cout << "Remaining balance in paise: "
                  << card.getBalance() << '\n';
    } else {
        std::cout << "Journey denied: minimum balance rule\n";
    }
}

std::cout << "Airport weekend fare in paise: "
          << calculateFare(Zone::Airport, true, false) << '\n';
}
```

OUTPUT

```
Fare paid in paise: 7200
Remaining balance in paise: 12800
Airport weekend fare in paise: 5400
```

The fourth Airport zone requires a new enum value and fare entry. The weekend discount is a separate policy and must not combine with the peak surcharge. A monthly pass should be represented by a Pass class containing the two selected zones and validity information; journeys in covered zones consume no stored-value fare while the pass is valid. The updated implementation demonstrates the changed fare design. For an Airport base fare of 6000 paise, a 10% weekend discount produces 5400 paise. This redesign is easier to extend when zone fares are stored in a single table, but validation must ensure the enum/index mapping is valid.

FINAL CONCLUSION

The fare system should use exact integer currency, strongly typed zones, centralized policy constants, validated control flow and encapsulated card balances. The supplied programs demonstrate the main conversion, precedence, control-structure, input, OOP and updated-design requirements. The GitHub repository link and screenshots of the student's own execution must be added before submission.